

Harness Engineering Coding Agent Guide

Autor: André Almeida

Visão geral

O Harness Engineering Coding Agent é uma skill operacional para orientar agentes de coding em trabalhos reais de software. Ela parte da premissa de que o principal gargalo da IA aplicada ao desenvolvimento não é apenas a geração de código, mas a criação de um ambiente de execução com contexto, especificação, sensores, critérios de aceite, segurança, release e aprendizado.

A skill foi desenhada para ser portátil. Ela pode ser usada por qualquer agente que consiga ler arquivos, editar código, executar comandos e registrar estado. Isso inclui agentes em IDEs, CLIs, ambientes autônomos, agentes internos de empresas e combinações multiagente.

A justificativa do método é prática, mas também acadêmica. Benchmarks de engenharia de software real mostram que modelos precisam compreender issues, navegar codebases extensas, coordenar alterações entre arquivos e validar mudanças em ambiente de execução.¹ Revisões sobre IA em IDEs indicam ganhos de produtividade, mas também registram riscos de **verificação insuficiente, dependência excessiva, manutenção, correção e segurança**.² A skill existe para responder exatamente a esse ponto: transformar agentes de coding em participantes de um processo verificável, e não em geradores isolados de código.

Código não é software. Software é um sistema sociotécnico que envolve intenção de produto, arquitetura, dados, segurança, testes, release, observabilidade, operação e evolução contínua.

O problema que a skill resolve

Agentes de coding tendem a ser muito bons em tarefas localizadas, mas podem falhar em tarefas reais quando recebem contexto insuficiente, objetivos vagos ou ausência de critérios objetivos de validação. O resultado costuma ser uma entrega que compila, parece funcionar em uma demonstração local, mas não está pronta para produção, manutenção ou evolução.

O harness resolve esse problema criando um conjunto de gates. Antes de codar, o agente entende a intenção. Antes de alterar estrutura, entende o contrato. Antes de declarar sucesso, roda sensores. Antes de lançar, planeja rollback e observabilidade. Antes de encerrar, deixa handoff.

Fundamentos acadêmicos e técnicos

A skill consolida oito linhas de evidência. Ela usa **SWE-bench** como alerta central: tarefas reais de software exigem raciocínio sobre issues, contexto de repositório, múltiplos arquivos e execução, não apenas geração de código em isolamento.¹ Ela usa a literatura sobre agentes LLM para engenharia de software para justificar loops de planejamento, uso de ferramentas, feedback, memória e execução iterativa.^{3 4}

Da engenharia de requisitos, a skill herda a necessidade de rastrear intenção, requisito, decisão, implementação, teste e handoff.⁵ De BDD e especificação comportamental, herda a prática de converter intenção de negócio em comportamento observável e validável antes do código.⁶ De runtime verification, SRE e observabilidade, herda a noção de que sistemas em produção precisam de sinais, invariantes, evidência, rollback e aprendizado operacional.^{7 8}

A camada de segurança é sustentada por DevSecOps, especialmente pela integração de práticas, ferramentas e métricas de segurança ao ciclo de desenvolvimento e operação.⁹ A camada humano-IA é sustentada por revisões de Human-AI Experience em IDEs, que recomendam transparência, explicabilidade, controle do usuário e auditoria para mitigar riscos de qualidade e dependência excessiva.² A camada mais recente é **Code as Agent Harness**, que trata o próprio código e os artefatos operacionais como infraestrutura versionável para raciocínio, ferramentas, memória, ambiente, verificação, telemetria, continuidade e aprendizado do agente.¹⁰

Fundamento	Tradução operacional na skill
SWE-bench e benchmarks de issues reais	O agente deve tratar a tarefa como alteração verificável em codebase, com ambiente e regressão.
Agentes LLM para engenharia de software	O fluxo usa planejamento, ferramentas, sensores, memória de estado e handoff.
Engenharia de requisitos e rastreabilidade	PRD, contrato, critérios de aceite, plano de teste e relatório de avaliação conectam intenção a evidência.
BDD e especificação comportamental	Behavior contracts e semantic specs tornam comportamento explícito antes da implementação.
Runtime verification, SRE e observabilidade	Release exige sinais, rollback, logs, métricas, replay quando necessário e evidência pós-lançamento.
DevSecOps	Mudanças sensíveis sobem o rigor de validação, revisão e aprovação.
Human-AI Experience	A skill preserva controle humano, reduz opacidade e exige auditoria do resultado gerado por IA.
Code as Agent Harness	O agente não apenas escreve código; ele opera dentro de um harness de artefatos versionáveis, sensores, memória de estado, telemetria, consentimento de atualização e melhoria sem regressão.

A versão consolidada dessas referências está em [academic-foundations.md](#).

Além da base acadêmica, a skill incorpora como inspiração operacional a skill pública [grill-me](#), que propõe entrevistar criticamente o usuário sobre um plano ou design até alcançar entendimento compartilhado. Na adaptação do Harness Engineering Coding Agent, essa ideia vira um gate mais disciplinado: o agente deve primeiro investigar o que o codebase já responde, depois registrar perguntas críticas em `DECISION_GRILL.md` e perguntar ao usuário apenas o que realmente bloqueia avanço seguro.

Arquitetura do pacote

Componente	Descrição
<code>SKILL.md</code>	Núcleo operacional. Define quando usar, como classificar tarefa, quais gates seguir e qual é a definição de pronto.
<code>references/product-engineering-discovery.md</code>	Ajuda o agente a transformar uma ideia vaga em intenção de produto validável.
<code>references/prd-specification-builder.md</code>	Converte intenção em PRD, contrato, requisitos, critérios de aceite e plano de teste.
<code>references/release-measurement-loop.md</code>	Organiza rollout, rollback, observabilidade, métricas e revisão pós-lançamento.
<code>references/version-check.md</code>	Define o protocolo de checagem da versão upstream, leitura do README público e consentimento antes de atualizar a skill local.
<code>references/squad-mode.md</code>	Define quando separar papéis e como coordenar um squad multiagente.
<code>templates/</code>	Modelos reutilizáveis para gerar artefatos no projeto-alvo.
<code>checklists/</code>	Lista prática para revisar a execução antes de encerrar a tarefa.
<code>examples/</code>	Exemplo mínimo de aplicação do fluxo.

Fluxo mental

O fluxo principal é:

Intenção de produto

- Descoberta de produto
- PRD / especificação / contrato
- Quebra em tarefas
- Implementação controlada
- Sensores de validação
- Release seguro
- Medição pós-lançamento
- Telemetria do harness
- Handoff e aprendizado

Esse fluxo não precisa ser executado inteiro em toda tarefa. A skill exige proporcionalidade. Uma correção simples deve continuar simples. Uma mudança que envolve dados, autenticação, produção ou cliente real não deve ser tratada como um snippet.

Classificação de tarefas

Tipo	Descrição	Processo recomendado
quick-fix	Correção simples, local e de baixo risco.	Git safety, alteração mínima e verificação objetiva.
feature	Novo comportamento ou interface.	Contrato, critérios de aceite, plano de teste e handoff.
product-feature	Feature com ambiguidade de produto.	Descoberta, PRD, contrato, critérios, teste e métricas.
architecture	Mudança estrutural ou decisão técnica relevante.	Arquitetura, decisão registrada, avaliação e handoff.
security-data	Auth, permissões, segredos, dados ou pagamentos.	Revisão mais rigorosa, critérios, testes e aprovação quando necessário.
release	Deploy, rollout, flags, rollback ou observabilidade.	Plano de release, sensores, rollback e medição.
investigation	Diagnóstico de bug, incidente ou comportamento incerto.	Estado, hipóteses, evidências e relatório de avaliação.

Modos de execução

O modo `solo` usa um único agente para executar o loop inteiro. É adequado para tarefas claras, pequenas e de baixo risco. O modo `review-pair` separa implementação e validação, sendo útil quando existe risco técnico ou de negócio. O modo `squad` separa papéis de produto, especificação, arquitetura, implementação, avaliação, release e registro, sendo indicado para tarefas longas, críticas ou ambíguas.

A decisão de modo deve considerar risco, escopo, impacto em produção, sensibilidade dos dados, ambiguidade e custo de coordenação.

Camada de especificação semântica

A análise do artigo sobre VibeCoding State-of-the-Art-Driven Development reforça um ponto importante: agentes de coding não devem ser usados apenas como geradores de boilerplate.

Em tarefas arquiteturais, agentic workflows, automações ou sistemas AI-native, o agente deve primeiro declarar o comportamento e as garantias do sistema.

A skill incorpora essa ideia por meio de `.harness/SEMANTIC_SPEC.md` e `references/semantic-system-design.md`. O objetivo não é obrigar o uso de BE2E, de uma DSL específica, de event sourcing ou de uma arquitetura hyper-polyglot. O objetivo é tornar explícitos comportamento, trigger, invariantes, garantias, constraints, eventos, estado, replay, auditabilidade, observabilidade, falhas e segurança antes da implementação.

Conceito do artigo	Adaptação na skill
IA como parceira de arquitetura	O agente deve fazer revisão arquitetural proporcional ao risco antes de decisões estruturais.
Uma declaração semântica antes de várias implementações	O template <code>SEMANTIC_SPEC.md</code> registra behavior contracts antes de arquivos e funções.
Runtime carrega complexidade	A skill usa artefatos repetíveis para reduzir complexidade operacional e evitar drift entre camadas.
Observabilidade, replay e auditabilidade	O release gate e os sensores passam a exigir evidência operacional quando o comportamento for crítico.
State-of-the-art-driven development	A referência de design semântico pede comparação entre padrões modernos e a opção mais simples suficiente.

Sensores

A skill diferencia sensores computacionais e sensores inferenciais. Sensores computacionais são testes, lint, typecheck, build, validação de schema, execução local, migração em dry-run, checks de acessibilidade e scans de segurança. Sensores inferenciais são revisões por IA, críticas de arquitetura, raciocínio de segurança e análise de cobertura de requisitos.

Sensores inferenciais são úteis, mas não substituem sensores determinísticos. Quando não houver teste automatizado disponível, o agente deve criar a menor verificação objetiva possível ou registrar claramente a lacuna.

Telemetria do harness

A telemetria do harness transforma o encerramento da tarefa em aprendizado operacional explícito. Ela não serve para coletar dados genéricos nem para justificar autoatualização silenciosa; serve para registrar, dentro dos artefatos do projeto, quais decisões, sensores, lacunas e evidências realmente sustentaram a entrega. Com isso, a próxima sessão não começa apenas com um resumo narrativo, mas com sinais verificáveis sobre o que foi testado, o que ficou incerto e quais melhorias do harness poderiam reduzir risco em trabalhos futuros.

Sinal de telemetria	O que deve ficar explícito	Artefato típico
Contexto utilizado	Quais arquivos, documentos, issues, requisitos ou decisões orientaram a execução.	STATE.md , EVALUATION_REPORT.md e HANDOFF.md .
Gates acionados	Quais gates foram necessários pela natureza da tarefa: produto, contrato, semântica, implementação, avaliação, release ou handoff.	STATE.md e relatório de avaliação.
Efetividade dos sensores	Quais testes, builds, lint, typecheck, inspeções ou revisões produziram evidência real.	TEST_PLAN.md e EVALUATION_REPORT.md .
Pontos cegos	O que não pôde ser validado, quais riscos permaneceram e quais hipóteses ainda dependem de verificação humana ou ambiente real.	EVALUATION_REPORT.md e HANDOFF.md .
Candidatos de melhoria	Quais ajustes futuros no harness poderiam melhorar rastreabilidade, validação, segurança, continuidade ou custo de coordenação.	HANDOFF.md , sem aplicar atualização automática.

Padrão de handoff

A skill exige handoff em todo trabalho significativo. O handoff deve permitir que outra pessoa ou outro agente continue sem reconstruir a sessão mental anterior. Um bom handoff informa estado atual, o que mudou, comandos executados, arquivos alterados, riscos, pendências e

próxima ação recomendada. Na versão atual, essa continuidade ficou mais explícita:

`STATE.md` deve preservar escopo aprovado, invariante atual, último estado verificado, links de evidência, notas de coordenação e ações pendentes inseguras; `HANDOFF.md` deve complementar esse registro com detalhes de continuidade, incluindo ponto exato de retomada, decisões já tomadas, lacunas abertas e recomendações para o próximo responsável.

Higiene de coding e atualização da própria skill

A evolução mais recente reforça duas disciplinas pequenas, mas importantes. A primeira é a higiene de coding: antes de editar, o agente deve pensar no objetivo e no raio de impacto; durante a implementação, deve escolher a solução mais simples suficiente; ao alterar arquivos, deve manter mudanças cirúrgicas; e antes de encerrar, deve verificar cada etapa com evidência objetiva ou registrar claramente a lacuna.

A segunda é a verificação de origem da própria skill. Em tarefas significativas, se houver acesso à internet, o agente pode consultar o repositório público, ler o `README.md`, comparar a cópia local e avisar o usuário quando houver mudança relevante. Essa rotina preserva controle humano: o agente pergunta antes de atualizar e nunca substitui a versão local em silêncio.

A terceira é a regra de **regression-free harness improvement**. Melhorar a própria skill não pode significar perder garantias já conquistadas. Toda atualização deve preservar rastreabilidade, sensores, critérios objetivos, continuidade de handoff, consentimento de atualização e salvaguardas de segurança. Quando uma simplificação for desejável, ela precisa ser justificada por evidência e não pode remover uma proteção sem substituí-la por alternativa equivalente ou melhor.

Como adaptar em ferramentas diferentes

Em agentes baseados em diretório de skills, copie o diretório inteiro da skill. Em agentes que usam regras, copie o conteúdo de `SKILL.md` como regra principal e mantenha `references/` e `templates/` acessíveis. Em agentes internos, trate `SKILL.md` como política de execução e os demais arquivos como módulos lidos sob demanda.

Histórico de alterações

Data	Hora	Motivo
2026-05-28	13:18 GMT-3	Atualização do guia para refletir a estrutura portátil, removendo dependência de subskills instaláveis e explicando o uso por qualquer agente de coding.
2026-05-29	05:54 GMT-3	Added semantic specification section, connecting VibeCoding state-of-the-art-driven development concepts to portable behavior contracts and proportional architecture review.
2026-05-29	06:00 GMT-3	Inclusão de seção de fundamentos acadêmicos e técnicos na introdução, conectando o storytelling da skill a benchmarks, literatura de agentes LLM, BDD, rastreabilidade, verificação, DevSecOps, SRE e Human-AI Experience.
2026-05-29	06:05 GMT-3	Inclusão de fundamentos acadêmicos consolidados e adaptação do padrão <code>grill-me</code> como decision grill para crítica de planos e designs.
2026-06-01	18:45 GMT-3	Inclusão de seção sobre higiene de coding e verificação da versão upstream da skill com consentimento explícito antes de atualização.
2026-06-04	12:26 GMT-3	Atualização do guia consolidado para incorporar Code as Agent Harness como fundamento acadêmico-operacional, telemetria do harness, continuidade aprimorada em <code>STATE.md</code> e <code>HANDOFF.md</code> e regra de melhoria sem regressão.

1. Jimenez, C. E. et al. [SWE-bench: Can Language Models Resolve Real-World GitHub Issues?](#), ICLR 2024.
2. Sergeyuk, A. et al. [Human-AI Experience in Integrated Development Environments: A Systematic Literature Review](#), Empirical Software Engineering, 2026.
3. Liu, J. et al. [A Survey on Large Language Model based Autonomous Agents for Software Engineering](#), ACM Transactions on Software Engineering and Methodology, 2025.
4. Yao, S. et al. [ReAct: Synergizing Reasoning and Acting in Language Models](#), 2022.
5. Nair, S. et al. [A survey on requirements traceability](#), Requirements Engineering, 2017.
6. Binamungu, L. P., Embury, S. M. and Konstantinou, N. [Characterising the Quality of Behaviour Driven Development Specifications](#), 2020.

7. D'Angelo, B. et al. [A survey of challenges for runtime verification from advanced application domains](#), Formal Methods in System Design, 2019.
8. Google. [Site Reliability Engineering](#), 2016.
9. Zhao, X., Clear, T., Lal, R. [Identifying the primary dimensions of DevSecOps: A multi-vocal literature review](#), Journal of Systems and Software, 2024.
10. Ning et al. [Code as Agent Harness](#), arXiv, 2026.